



Google Summer of Code Proposal 2026

BLT University

Introduction

Personal Details:

Name	Sakshee Suman
Github	e-esakman
Email	–
Linkedin	Sakshee-suman04
University	Dr. A.P.J. Abdul Kalam Technical University
Timezone	Indian standard time (UTC+5:30)
Contact	–

Project Details:

Project name: Blt-University - security focused platform

Project size: Large

Project duration: 350 hr

Mentors Name: Abhishek Kumar, Aryan jain, Donnie Brown

Introduction.....	1
Personal Details:.....	1
Project Details:.....	1
 About Me.....	 2
Motivation.....	3
Contributions to BLT.....	4
Merged prs.....	4
Reviewed Pr's.....	4
 Synopsis.....	 4
Benefits to the community.....	5
Current status of the project:.....	6
 Goals:.....	 6
Deliverables:.....	7
Testing Plan:.....	18
Success Metrics:.....	20
Risks & Mitigations:.....	21
 Approach:.....	 22
System Architecture:.....	22
 Timeline.....	 24
Timeline Detailed.....	24
Community Bonding Period (May 1 – May 24).....	24
Week 1 (May 25 – May 31).....	25
Week 2 (June 1 – June 7).....	26
Week 3 (June 8 – June 14).....	26
Week 4 (June 15 – June 21).....	26
Week 5 (June 22 – June 28).....	27
Week 6 (June 29 – July 5).....	27
Midterm Evaluation (July 6 – July 10).....	27
Week 7 (July 6 – July 12).....	27
Week 8 (July 13 – July 19).....	28
Week 9 (July 20 – July 26).....	28
Week 10 (July 27 – August 2).....	28
Week 11 (August 3 – August 9).....	28
Week 12 (August 10 – August 16).....	29
Final Phase (August 17 – August 24).....	29
Timeline Summary.....	29
 Commitments.....	 35
 Post GSoC.....	 35
 LLM Usage.....	 35

ABOUT ME

I am a pre-final year Computer Science student with interests in web development, open-source contribution, and application security. I have experience working with Python, Django, JavaScript, and frontend technologies, and I have been actively contributing to the OWASP BLT ecosystem.

Through my contributions, I have worked across UI, backend logic, and data pipelines, improving system reliability by fixing critical issues such as data fetching failures ([#191](#)) and runtime errors ([#5983](#)), contributing to system migrations ([#14](#)), and strengthening validation and error handling ([#5392](#)). I have also worked on security-sensitive areas, including resolving Markdown parsing vulnerabilities ([#5634](#)) and improving the secure handling of external content in blog fetching ([#5897](#)).

Through reviewing contributions in the BLT repository, I developed the ability to identify recurring patterns such as validation gaps, unsafe data handling, N+1 queries, and inefficient data access.

These experiences shaped my understanding of how security issues emerge in real-world systems and directly inspired the idea of converting such issues into structured learning labs in BLT-University.

Motivation

I discovered open source through OWASP BLT, and it completely changed how I approached development. What started as curiosity exploring the repository and understanding how contributions work quickly turned into active participation. I began creating pull requests, reviewing code, and engaging deeply with the project.

One of the most impactful parts of this journey has been BLT's peer review environment. Every pull request goes through discussion and feedback, which taught me how to spot issues, think critically about code, and improve not just my own contributions but also others'. This experience helped me understand that writing code is only one part of development. Understanding *why something is wrong* and how to **fix it securely** is equally important.

While contributing, I noticed that many fixes are made without fully **understanding the underlying security implications**. This creates a gap between fixing issues and actually learning from them. This realization led me to this proposal.

I want to build something that helps contributors not just solve problems, but truly understand them. Through the issues I've worked on and the patterns I've observed, I aim to build a platform that not only teaches vulnerabilities but also helps contributors develop a mindset for identifying and fixing security issues in real-world applications.

BLT University provides the perfect opportunity to do this combining open source, security, and interactive learning into one platform. My goal is to help contributors become more confident, thoughtful, and security-aware, so they can contribute better to the open source community.

Contributions to BLT

I have been actively contributing to the OWASP BLT ecosystem since (Nov/2025), focusing on improving platform stability, feature enhancements, reviewing other pull requests and security-related workflows. Through these contributions, I have developed a strong understanding of the repository's structure, development workflow, and architectural decisions.

Merged prs

(25 + pr's merged)

Some core contributions

- [#15](#) - feat(ui): Introduce BLT-University UI inspired by BLT design.
- [#50](#) - feat: Add bulk PR import from repository with integrated search functionality.
- [#5826](#) - chore: Remove blog module from core and refactor related dependencies.
- [#191](#) - fix: Resolve "Failed to fetch PR" data retrieval error.
- [#4807](#) - fix: Resolve map zoom and highlight issue for banned countries.
- [#5494](#) - fix: Improve error messaging and UI on 500 error page
- [#5392](#) - fix: Improve form validation, comment error handling, and filter logic.
- [#5634](#) - fix: Resolve Markdown parsing errors.
- [#14](#) - feat: Migrate time log manager dashboard from Sizzle to BLT-Fresh.
- [#151](#) - fix: Correct toggle state handling logic.
- [#5231](#) - fix(ui): Enhance email password reset pages and discussion room creation UI.
- [#5349](#) - fix: Enable users to delete discussion rooms.
- [#5185](#) - fix: Implement secure user account deletion flow.
- [#5897](#) - fix: secure dev.to blog fetching and display to home page.
- [#5983](#) - bug fix: TypeError 'list' object is not callable in settings.py

Reviewed Pr's

40+ pr's reviewed

Synopsis

BLT university is evolving into a GitHub Pages–based, static cybersecurity education designed to provide open, collaborative, and version-controlled learning resources.

This project proposes the introduction of a hands-on, code-centric security lab framework that turns BLT University into a platform where contributors can actually practice identifying and fixing security issues instead of only reading about them.

While contributing to BLT, I noticed that contributors often fix issues without fully understanding the underlying security implications.

The project transforms BLT-University into an interactive platform by introducing:

- A structured lab engine (Identify → Explain → Fix) with a semantic answer-validation engine that accepts correct fixes regardless of variable naming or whitespace differences.
- A vulnerability intelligence dashboard based on real PR data.
- An adaptive lab recommendation system backed by a concrete, parameterised scoring formula.

The goal is to help contributors build a habit of thinking about security while writing code, using real examples from BLT. BLT University can help contributors develop stronger security awareness while improving the quality of contributions to open-source projects.

Benefits to the community

This project directly supports BLT’s core mission of building security-aware contributors in a sustainable open-source ecosystem.

The implementation will provide:

Structured Security Labs - Instead of just static reading material, contributors will work through guides which will help them to identify vulnerability and fix it. It will help analyze vulnerable code, reason about how it can be exploited, and apply secure fixes step by step and common vulnerability patterns observed in BLT issues and pull requests will be anonymized and transformed into structured labs and remediation guides.

Intelligence Dashboard - Aggregated data from PR review comments shows which vulnerability categories recur most, directing learners to the most impactful labs first.

Semantically-Aware Validation - The previous lab implementation relied on strict string matching, which rejected correct solutions if variable names, formatting, or approach differed. To fix this, I will introduce a more flexible validation system that focuses on how the fix works rather than how it is written. This allows multiple correct implementations (e.g., different variable names or equivalent secure patterns) to be accepted, making the labs more practical and usable.

Example use case:

“A contributor introduces print() statements while fixing an issue in the repository. They later recognize the risk of exposing sensitive request data and replace them with proper logging.”

Current status of the project:

- BLT University currently hosts static educational content on GitHub Pages.
- The platform UI has already been developed and integrated into the BLT ecosystem.
- Existing security labs are available in the OWASP-BLT Django application and will be migrated to BLT-University.
- Interactive lab workflows and intelligence-based learning insights are currently missing.
- This project introduces a Markdown-based lab system, interactive security reasoning workflows, and intelligence-driven insights to improve contributor security awareness.

Also, the primary source of lab data is the OWASP-BLT main repository, which is built using Django. Security labs are stored in three core models:

- Labs: stores metadata such as name, description, estimated_time, and order
- Tasks: defines individual steps with **task_type** (theory or simulation)
- TaskContent: contains theory content (HTML), MCQs, and simulation configurations (JSON)

Currently, these labs are rendered as HTML pages and are not exposed via a REST API.

Goals:

The primary goal of this project is to transform BLT University into a structured and interactive cybersecurity learning platform.

The project focuses on eight key objectives:

1. Interactive Lab Framework - Develop an interactive lab engine that guides users through structured security reasoning using the Identify → Explain → Fix workflow.
2. Static Lab Authoring System- Labs are created directly in _labs/ as Markdown + YAML, making contributions simple. Existing BLT labs are ported once as static files, removing any backend dependency.
3. Security Intelligence Integration - Analyze contributor mistakes using GitHub issues and pull request discussions from the BLT repository. Code review comments (including CodeRabbit) are used as a filtered supplement to identify recurring security patterns (e.g., SQLi, XSS, IDOR). The system focuses on identifying consistent patterns rather than relying on predefined labels or structured vulnerability tags.
4. GitHub Actions Validation Pipeline - Automated validation workflows that ensure newly added labs follow the required structure and schema before being merged.

5. Version-Controlled Lab Repository - A collection of Markdown-based labs organized by vulnerability category and difficulty level, enabling collaborative editing through pull requests.
6. Security Intelligence Dashboard - A visual dashboard displaying aggregated trends such as common vulnerability types and frequently observed security mistakes.
7. AI-powered Learning Assistant - An AI-powered learning assistant that provides context-aware hints based on active lab content, with a static fallback for offline use.
8. Interactive Learning Tracks - A Professional Track for companies and security teams (real-world PR-based vulnerability scenarios, minimal hints, advanced reasoning), and a Cyber Hygiene Track for everyday users learning good digital security habits (phishing, passwords, safe browsing — no coding required).

Deliverables:

The following deliverables will be completed during the GSoC project. These deliverables collectively transform BLT University into a structured, interactive cybersecurity learning platform built on a static GitHub Pages architecture.

1. Static Lab Authoring System

Labs live directly in the `_labs/` directory of the BLT-University GitHub repository as Markdown files with YAML frontmatter. Each file is a self-contained lab — no backend, no database, no API calls needed.

- Each lab file (e.g., `_labs/sql-injection-basics.md`) has YAML frontmatter defining: `title`, `category`, `difficulty`, `owasp_category`, `track` (professional | hygiene | both), and `steps` (an array of step objects with type, content, and validation config).
- Existing BLT labs are ported **once** manually as static Markdown files during the Community Bonding period — a one-time authoring effort, not an ongoing sync.
- Jekyll reads the `_labs/` collection at build time (registered in `_config.yml`) and renders each lab using `_layouts/lab.html`, which loads the client-side lab engine.
- New labs can be added by any contributor simply by writing a Markdown file and opening a pull request.
- A `validate-labs.yml` GitHub Actions workflow runs on every PR touching `_labs/`, parsing YAML frontmatter with `python-frontmatter` + `jsonschema` and blocking merges on schema errors.
- No Django dependency, no REST API endpoint, no export script needed.
- A Jekyll generator plugin automatically builds `_data/labs.json` at build time from the `_labs/` collection, enabling client-side search and dynamic filtering.

Example Markdown lab format:

```

1 ---
2 layout: lab
3 title: "Insecure Direct Object Reference (IDOR)"
4 slug: a01-idor
5 category: "Access Control"
6 difficulty: beginner
7 owasp_category: "A01:2021-Broken Access Control"
8 estimated_time: 45
9 description: >
10     Master IDOR vulnerabilities where applications expose internal
11     object references without proper authorization checks.
12
13 tasks:
14   - name: "Introduction to IDOR"
15     type: theory
16     order: 1
17
18 last_synced: "2026-05-01T02:00:00Z"
19 source_id: 1
20 ---

```

Migration Strategy: One-Time Lab Porting

- A one-time migration process will be used to convert existing lab content into Markdown-based modules compatible with BLT-University.
- A Python-based utility will convert existing lab data into Jekyll-compatible Markdown files with YAML frontmatter.
- This process is executed only during the initial setup phase and is not part of the runtime system.
- After migration, all lab content resides directly in the `_labs/` directory, making the platform fully independent of any backend or external system.
- All future labs will be authored directly via pull requests, making the repository the single source of truth.

Lab Migration Script

I'll write a `port_blt_labs.py` script that reads from the BLT repository via GitHub API, converts each lab's frontmatter to the new schema, and writes it to `_labs/`.

```

1 import os, base64, requests, frontmatter
2 from pathlib import Path
3
4 TOKEN = os.environ["GITHUB_TOKEN"]
5 REPO = "OWASP-BLT/BLT"
6 HEADERS = {"Authorization": f"token {TOKEN}"}
7
8 def fetch_labs():
9     url = f"https://api.github.com/repos/{REPO}/contents/_courses"
10    return requests.get(url, headers=HEADERS).json()
11
12 def transform(post, slug):
13     meta = post.metadata
14     return {
15         "title": meta.get("title", slug.replace("-", " ").title()),
16         "category": meta.get("category", "general"),
17         "difficulty": meta.get("difficulty", "beginner"),
18         "owasp_category": meta.get("owasp_category", "A03:2025"),
19         "track": meta.get("track", "both"),
20         "steps": meta.get("steps", []),
21         "migrated_from": f"{REPO}/_courses/{slug}.md",
22     }
23
24 def port(file):
25     data = requests.get(file["url"], headers=HEADERS).json()
26     raw = base64.b64decode(data["content"]).decode()
27
28     post = frontmatter.loads(raw)
29     slug = Path(file["name"]).stem
30
31     out = frontmatter.Post(post.content, **transform(post, slug))
32     Path("_labs", f"{slug}.md").write_text(frontmatter.dumps(out))
33
34     print(f" {slug}")
35
36 if __name__ == "__main__":
37     labs = [f for f in fetch_labs() if f["name"].endswith(".md")]
38     for lab in labs:
39         port(lab)
40
41     print(f"\nDone: {len(labs)} labs migrated")

```

2. Interactive Lab Framework

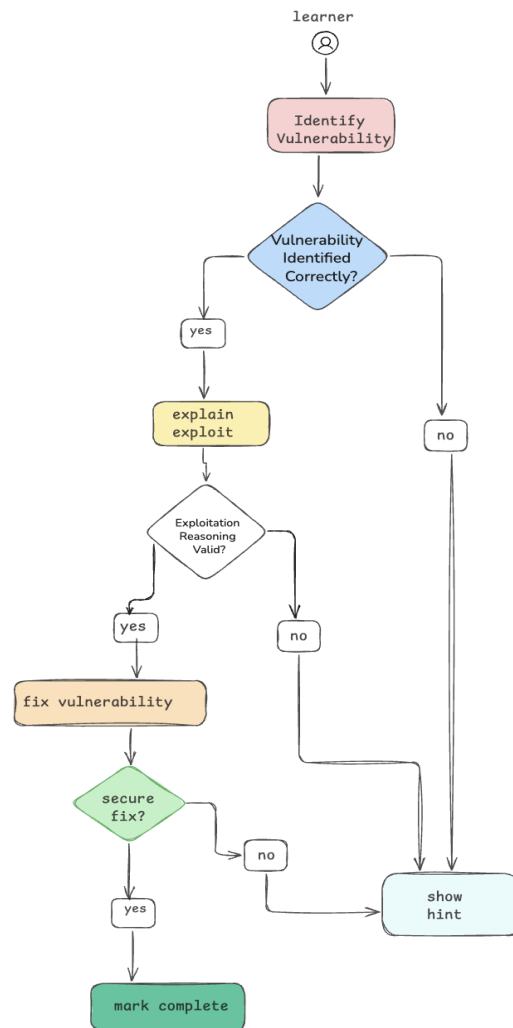
2a. Lab Engine (Identify → Explain → Fix)

A JavaScript-based state machine walks users through three phases:

- Identify: click/highlight the vulnerable line(s) from a code snippet.
- Explain: answer an MCQ or short explanation question about why the code is vulnerable.
- Fix: submit a corrected code snippet in an inline editor.

Hint Behaviour & Retry Flow

Hints are progressive guidance. When a learner triggers a hint, they receive a contextual clue that helps them understand the problem better, but the step is not auto-completed. After reading the hint, the learner retries the same step — and their updated submission is scored normally.



2b. Flexible Answer Validation

The previous lab system rejected correct fixes if they didn't exactly match the expected answer (for example, different variable names or formatting). This made the labs unreliable.

To fix this, I'm focusing on validating whether the vulnerability is actually fixed, instead of checking for a single exact solution.

The validation runs in the following order:

Step 1 → Structural check (code structure based)

The submitted code is converted into a structured representation (AST), which ignores variable names and formatting.

JavaScript-based labs will use the Acorn parser to generate ASTs in the browser, while Python-based labs will use the built-in AST module executed via Pyodide.

This enables validation of the logical structure of the fix, ensuring that semantically correct solutions are accepted even if implemented differently.

Step 2 → Behaviour-based validation

If the structure doesn't match (for example, a different valid approach), the code is tested using predefined attack cases.

If the attack still works → the fix is incorrect

If the attack fails → the fix is accepted

Step 3 → Pattern-based checks (fallback)

For cases where parsing or execution is not possible (e.g. HTML or configs), simple rules are used.

For example:

- detecting parameterized queries
- checking for input sanitization

To keep the scope manageable, the initial implementation will focus on common vulnerabilities like SQLi and XSS, and expand later based on feedback. This approach ensures that different valid solutions are accepted while still verifying that the vulnerability is actually resolved.

2c. Scoring System

Validation checks *whether* a fix is correct. Scoring measures *how well* the learner performed. These are intentionally separate.

- Scoring is fully **rule-based** - deterministic, lightweight, and OWASP-aligned so results are consistent and predictable.
- The evaluation operates in a constrained lab environment where expected concepts are predefined, ensuring that rule-based matching remains reliable and context-aware.

Step	Scoring Method
Identify	Exact match on selected vulnerable line(s); partial credit for partially correct selections
Explain	Concept-based evaluation using predefined security patterns (e.g., presence of terms related to input sanitization, parameterized queries, or access control checks), supported by rule-based matching to ensure consistent and deterministic scoring.
Fix	Diff-based comparison of submitted patch against reference fix; AST equivalence check for structural matches

2d. Execution Model

Lab Type	Execution	Reason
Theory / MCQ	Fully Static	No code execution required
Fix based labs	Client-side validation (AST + predefined test cases)	Fast, works entirely in browser, no backend required
Python Labs	Pyodide WASM - runs in the browser.	Full Python execution client-side, no server needed
Edge Cases	Optional Cloudflare Python Worker	Only if client-side validation is insufficient; system works 100% without it

This model avoids unnecessary server infrastructure while still supporting realistic validation where required. The system is fully functional without Cloudflare Workers, they are an optional enhancement only for tasks that genuinely cannot be done client-side (e.g., proxying AI hint requests to keep API keys server-side).

Sandbox Compatibility: BLT's existing sandbox components (Django-rendered) are reimplemented using Pyodide ($\geq 90\%$ of cases) or sandboxed iframes.

Cloudflare Workers are not part of the default architecture.

They are only introduced for specific edge cases where client-side execution is insufficient.

I will start by focusing on a small working set (such as SQLi and XSS labs) to ensure the system works reliably. The scope can be expanded later based on mentor feedback.

3. GitHub Actions Validation Pipeline

A validate-labs.yml CI workflow runs on every pull request that touches `_labs/`:

- Parses YAML frontmatter with `python-frontmatter + jsonschema`.
- Validates that `validation_strategy` is one of the allowed values.
- Checks that `test_cases` are present when `validation_strategy: test_cases`.
- Fails the PR with line-specific error messages if any lab file is malformed.


```

1  name: Validate Security Labs
2
3  on:
4    pull_request:
5      paths:
6        - "labs/*.md"
7
8  jobs:
9    validate:
10     runs-on: ubuntu-latest
11
12     steps:
13       - uses: actions/checkout@v3
14
15       - name: Install dependencies
16         run: pip install jsonschema
17
18       - name: Validate Lab Structure
19         run: python scripts/validate_labs.py

```

4. Security Intelligence Data Pipeline

The pipeline extracts recurring security patterns from GitHub issues, pull requests, and review comments using a structured, rule-based approach designed for reliability and interpretability.

It is implemented as a **CI-driven data pipeline** that runs periodically and generates a static dataset consumed by the dashboard.

4a. Data Collection

GitHub REST API is used to fetch:

- Issues
- Pull request discussions
- Review comments (including CodeRabbit where relevant)

Data is stored temporarily as structured JSON for processing.

4b. Noise Filtering (Preprocessing)

Most comments are not security-related (e.g., formatting, naming, refactoring).

To reduce noise:

- Comments containing terms like “typo”, “indentation”, “refactor” are filtered out.
- Only comments with potential code behavior or input/output context are retained.

4c. Pattern-Based Security Detection (Core Logic)

The system detects security-relevant patterns using rule combinations.

Each rule checks for:

- Source indicators (e.g., input, request, params)
- Sink indicators (e.g., query execution, rendering, database access)
- Missing safeguards (e.g., no sanitization, no parameterization)

Example rule:

IF (input/request present) AND (query/execute present) AND NOT (parameterized/prepared)
→ classify as Potential SQL Injection

This approach reduces false positives and captures actual vulnerability intent instead of isolated keywords.

Lightweight NLP normalization

To improve matching consistency, I will normalize tokens using stemming (e.g., via **nltk** in Python).

This ensures variations like “sanitize”, “sanitized”, and “sanitization” are treated as the same concept during rule evaluation.

Confidence scoring

Each detected pattern is assigned a `confidence_score` based on:

- number of matched signals (source, sink, missing safeguard)
- rule strength

Only patterns above a defined threshold (e.g., 0.7) are included in the final dataset.

Lower-confidence matches are treated as unclassified and handled via fallback.

4d. Output Generation

Processed results are stored as:

_data/vulnerability_intel.json

Structure includes:

- category counts (SQLi, XSS, IDOR, etc.)
- timestamps
- optional trend data

This file is consumed directly by the dashboard.

4e. Baseline Dataset Fallback (Core Component)

To ensure the system always has meaningful data, a baseline dataset (`_data/seed_intel.json`) is included as a core component of the pipeline.

This dataset is used:

- during initial deployment (before sufficient real data is collected)
- when extracted data is sparse or low-confidence

The CI pipeline dynamically decides whether to use real extracted data or fall back to the baseline dataset.

This ensures the system is never dependent on real-time data availability and remains stable.

These patterns are used for learning insights and do not represent confirmed security vulnerabilities.

Optional integration with NetGuardian (future enhancement)

The pipeline can optionally integrate with outputs from OWASP BLT-NetGuardian, which provides real vulnerability findings from automated scanning systems.

This allows:

- validation of detected patterns against real-world vulnerabilities
- mapping between contributor mistakes and actual discovered vulnerabilities

This integration is not required for the initial implementation but provides a strong extension for improving dataset quality and real-world relevance.

5. Vulnerability Intelligence Dashboard

A visual dashboard will present aggregated insights from contributor activity.

This includes:

- The dashboard reads `_data/vulnerability_intel.json` (generated in CI) and renders it using Chart.js. It displays:
 - Bar chart : top 5 vulnerability categories by frequency.
 - Line chart : month-over-month trend for the top 3 categories (if historical data is available).
 - Pie chart : percentage breakdown of all classified categories.
 - The dashboard will show when the data was last updated so users know how recent it is.
- Charts will load smoothly and adjust properly when the screen size changes.
- Community data will come from a pre-built JSON file, while user progress will be stored separately in localStorage and will not be mixed with shared data.

6. Adaptive Lab Recommendation System

The recommendation system suggests labs based on user progress and real-world relevance.

Each lab is scored using:

$$\text{score}(\text{lab}) = 0.4 \times \text{freq_norm} + 0.4 \times \text{incompleteness} + 0.2 \times \text{owasp_weight}$$

The recommendation system is designed to guide learners toward the most relevant labs based on both real-world security trends and their individual learning progress.

Instead of suggesting labs randomly, the system prioritizes:

- vulnerabilities that appear frequently in real BLT contributions, and
- areas where the learner has not yet developed understanding.

Factors used:

Factor	Meaning	Formula
--------	---------	---------

incompleteness	how much of the lab is unfinished	$\text{incompleteness} = 1 - \text{completion_rate}$
freq_norm	how common the vulnerability is in PR data	$\text{freq_norm} = \text{count} / \text{max_count}$
owasp_weight	priority based on OWASP Top 10 ranking	Mentioned in table below

OWASP Top 10 2025 weight table (used as **owasp_weight** in the formula, stored in **_data/owasp_weights.json**):

OWASP ID	Category	2025 Rank	Owasp Weight
A01:2025	Broken Access Control	1	1.00
A02:2025	Cryptographic Failures	2	0.95
A03:2025	Injection	3	0.90
A04:2025	Insecure Design	4	0.85
A05:2025	Security Misconfiguration	5	0.80
A06:2025	Vulnerable & Outdated Components	6	0.75
A07:2025	Identification & Authentication Failures	7	0.70
A08:2025	Software & Data Integrity Failures	8	0.65
A09:2025	Security Logging & Monitoring Failures	9	0.60
A10:2025	Server-side Request Forgery	10	0.50

Not started → incompleteness = 1.0

Completed → incompleteness = 0

Higher score means higher recommendation priority.

Example:

freq_norm = 1.0

incompleteness = 0.8

owasp_weight = 0.9

score = $0.4 \times 1.0 + 0.4 \times 0.8 + 0.2 \times 0.9 = 0.4 + 0.32 + 0.18 = 0.9$

Note: incompleteness is derived from localStorage, so it is device-specific. If a user switches devices, they will appear as a new user until progress is rebuilt locally. Progress is stored locally for now to keep things simple with the current static setup. It can be extended later once backend support is available.

Top-ranked labs are shown as recommendations.

The weights are intentionally simple and can be adjusted later based on mentor feedback.

Each factor captures a different aspect:

- freq_norm reflects real-world relevance, the values are computed from aggregated PR data during each pipeline run and remain fixed for the generated dataset.
- incompleteness reflects user progress, rather than just marking a lab as done or not, the system keeps track of which step the learner gets stuck on.
- owasp_weight reflects security priority

This keeps the system simple and interpretable, while still capturing both user learning needs and real-world vulnerability importance.

SCOPE REGARDING BADGES

Badges awarded for completing OWASP categories are client-side visual indicators stored in localStorage. They are not connected to BLT's backend UserBadge model or BACON token system. The system is designed to support future integration with BLT rewards via APIs once authentication and backend support are available, but this integration is intentionally kept out of scope for the current project to maintain a fully static and feasible architecture.

7. AI-Powered Learning Assistant

A lightweight AI assistant will provide:

- Context-aware hints based on lab content.
- If AI services are unavailable, a static FAQ-based fallback system will provide predefined hints and explanations.
- Model: The system can use any lightweight LLM (e.g., OpenAI gpt-4o-mini, Google Gemini 1.5 Flash, or similar free-tier models). The architecture allows easy switching between providers without affecting the frontend.

8. Learning Tracks

To make the platform useful for different kinds of users, I'm dividing it into two learning paths.

1. **Professional Security Track:** This track is designed for developers and contributors who already work with code. The main idea is to teach security using real examples instead of simplified or unrealistic scenarios. The labs are based on actual pull requests

in BLT as well as common vulnerability patterns from the OWASP Top 10, so the learner is working with issues that reflect both real projects and widely recognized security risks.

Each lab follows a simple and consistent flow. The learner starts by reading a piece of code or a diff and identifying where the issue exists. This step helps build the habit of spotting problems instead of being directly told where they are. After that, the learner explains the issue in their own words, ensuring they understand what is wrong and why it is risky.

Once the problem is clear, the learner writes a fix directly in an inline editor. The system then checks whether the vulnerability is actually fixed. It does not depend on matching one exact answer. If the learner solves the problem in a different but correct way, it is still accepted. This reflects real development, where there can be multiple valid fixes.

This track will be made also suitable for organizational use, where companies can use it to train developers in secure coding practices. Teams can use these labs as part of onboarding or internal security training to help engineers identify and fix vulnerabilities in real-world code.

- Labs are based on real BLT pull requests and real vulnerability patterns
- Focus is on Identify → Explain → Fix, not just solving blindly
- Validation checks the outcome of the fix, not exact wording or structure
- Multiple correct solutions are accepted

Overall, the goal of this track is to help learners build a security mindset while working with code, not just complete tasks.

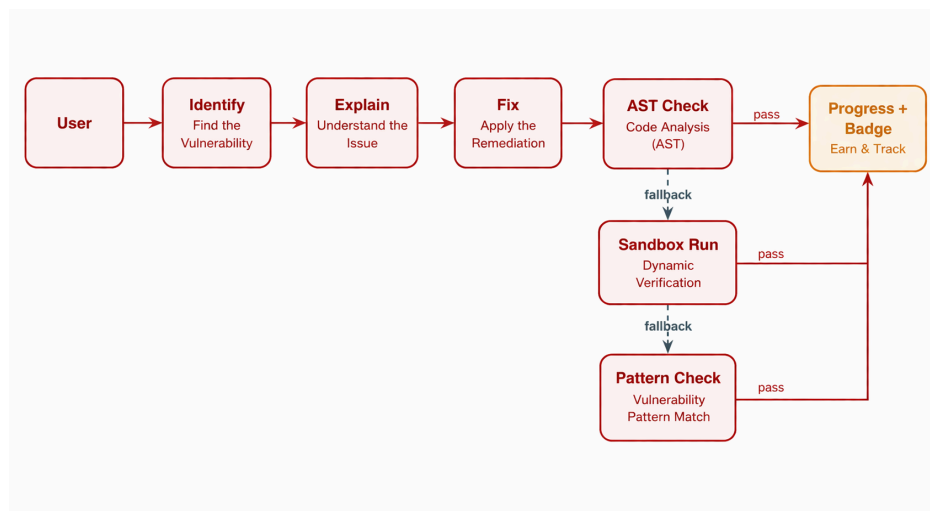
2. **Everyday Security Track:** This track is designed for non-developers who still face security risks in everyday life, such as suspicious emails, weak passwords, or confusion around things like two-factor authentication.

Instead of quizzes or long explanations, it uses realistic, interactive scenarios. The learner is placed in situations like a phishing email that looks like it's from their bank and can explore it naturally by checking the sender, reading the message, or clicking the link.

If they click the link, they see a fake login page and learn what signs they missed and what data could be exposed. If they inspect the email carefully, the system highlights what looks suspicious, such as mismatched domains or urgent language.

Every action shows what happens next, and even mistakes are used to explain consequences and correct responses.

Other scenarios include password safety, scam calls, two-factor authentication, and public WiFi decisions. The focus is on learning through real situations so users can apply it in everyday life.



UI MOCKUP : [FIGMA Prototype](#)

MODULE 04 — Web Vulnerabilities

SQL Injection: The Login Bypass

Understand how unsanitized user inputs can compromise your database security through architectural flaws in query construction.

Source Context
Security Audit Flag: PR #5897

Issue: Unsanitized user input in login flow

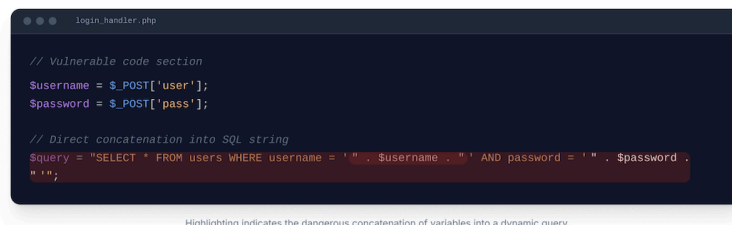
Severity: **CRITICAL**

The security team has identified a potential entry point for SQL injection within the authentication module. User-provided username and password fields are being concatenated directly into the query string without validation or parameterization.

auth-service.php

v1.2.4-stable

01. Identify the Vulnerability



Understand the Issue

By inputting special characters like `' OR '1'='1'`, an attacker can manipulate the logic of the SQL statement. This payload forces the query to return true for every row, potentially logging them in as the first user in the database—typically the administrator.

- ⊗ Missing input sanitization
- ⊗ Failure to use prepared statements

WHY THIS MATTERS

Impact: Authentication Bypass

A successful exploit grants full access to protected accounts without needing valid credentials. This leads to data breaches, unauthorized privilege escalation, and total system compromise.

Testing Plan:

Since this project involves multiple moving parts (data pipelines, UI interactions, and client-side logic), I'll focus on testing each layer separately to make sure everything works reliably together.

1. Static Lab Content Validation (pytest + jsonschema)

Every lab file in `_labs/` is validated against the YAML schema before merging.

- Parse all `_labs/*.md` files using `python-frontmatter`
- Validate required fields: `title`, `category`, `difficulty`, `owasp_category`, `track`, `steps`
- Ensure `validation_strategy` is one of the allowed values
- Fail CI with line-specific error messages on schema violations

2. Static Fetch Tests

Verify that labs load correctly from the `_labs/` collection via GitHub Pages.

- Validate that YAML frontmatter data is correctly embedded in rendered HTML
- Verify all lab pages are generated by Jekyll build (no 404s).
- Test that the static JSON index (`_data/labs.json`) auto-generated by a Jekyll generator plugin at build time matches all files in `_labs/`
- Verify dynamic search uses `_data/labs.json` correctly (no 404, correct structure)

3. Lab Engine Testing (Jest)

The lab engine will be tested to ensure correct state transitions and user interactions.

- Validate Identify → Explain → Fix flow
- Ensure incorrect actions show hints without breaking flow
- Verify progress is stored correctly in localStorage
- Test AST-based fix validation with correct and incorrect submissions

4. Pyodide Execution Tests

- Verify Python labs execute correctly in Pyodide WASM environment
- Test that correct fixes pass and vulnerable snippets fail
- Validate execution isolation (no access to browser globals outside sandbox)
- Test in headless browser environment (Playwright)

5. Cross-Browser & End-to-End Testing (Playwright)

End-to-end tests will verify that the complete system works correctly from a user's perspective.

- Verify lab pages load correctly from `_labs/` Markdown files
- Ensure UI components render properly across screen sizes

- Validate dashboard charts display correct data
- Test lab interaction (Identify → Explain → Fix flow)
- Validate navigation across pages
- Ensure consistent UI behavior (buttons, transitions, states)

6. CI Integration (GitHub Actions)

Continuous Integration will be used to automatically run tests and validate changes on every push or pull request so issues are caught before they reach the main branch.

- Validate YAML frontmatter for all `_labs/*.md` files
- Verify Jekyll build completes without errors
- Verify no broken builds are introduced

Additional edge cases tested:

- Incorrect but syntactically valid fixes
- Partial fixes where vulnerability still exists
- Empty or malformed lab submissions

Success Metrics:

Labs migrated to <code>_labs/</code>	100% migration of existing labs (currently 10)
Schema validation pass rate	≥ 95 % of all lab PR submissions
Security-related review records classified	20–50 classified patterns depending on dataset size. If fewer than 20 patterns are identified, a <code>dashboard_notice</code> flag will inform users, and the actual count will be reported during midterm evaluation.
Interactive lab engine vulnerability types	≥ 4 (SQLi, XSS, IDOR, Broken Auth)
Dashboard chart types rendered	≥ 3 (bar, line, pie)
AI assistant fallback coverage	≥ 90 % of lab steps covered by <code>_data/hints.json</code>
Cross-browser lab engine tests	Passing on Chrome, Firefox, Safari
Export pipeline CI runtime	< 2 minutes

Risks & Mitigations:

RISK	LIKELIHOOD	IMPACT	MITIGATION
GitHub REST API rate limit	Low	Medium	Use a dedicated GitHub Actions token; paginate requests; cache responses in <code>_data/</code> .
Static site limitations	Low	Low	Everything will run on client-side JS + JSON. No backend dependency, so it works fully on GitHub Pages.
Insufficient security data from CodeRabbit	Medium	Medium	Combine GitHub issues and PR discussions with filtered review comments. Use pattern-based filtering and fallback logic to avoid misleading insights.
Invalid lab format breaking site	Low	High	<code>validate-labs.yml</code> CI workflow blocks merges on schema errors before they reach the build.
localStorage data misuse	Low	Low	I'll only store basic progress (no sensitive data), so even if accessed, it's harmless.
Pyodide load time	Low	Low	Load Pyodide only when required for Python-based steps. Show loading indicator; cache WASM binary after first load.
Client-side AST validation edge cases	Medium	Medium	Start with a small working set (SQLi, XSS); expand coverage iteratively based on mentor feedback. Fallback to pattern-based checks.

Approach:

System Architecture:

The architecture is static-first and repository-driven.

All labs are stored directly in the `_labs/` directory as Markdown files with YAML frontmatter.

These labs are rendered through Jekyll into a static GitHub Pages site.

Existing labs from the OWASP-BLT Django system are ported once during the initial setup phase and are no longer fetched dynamically.

The system does not depend on any backend API or database at runtime.

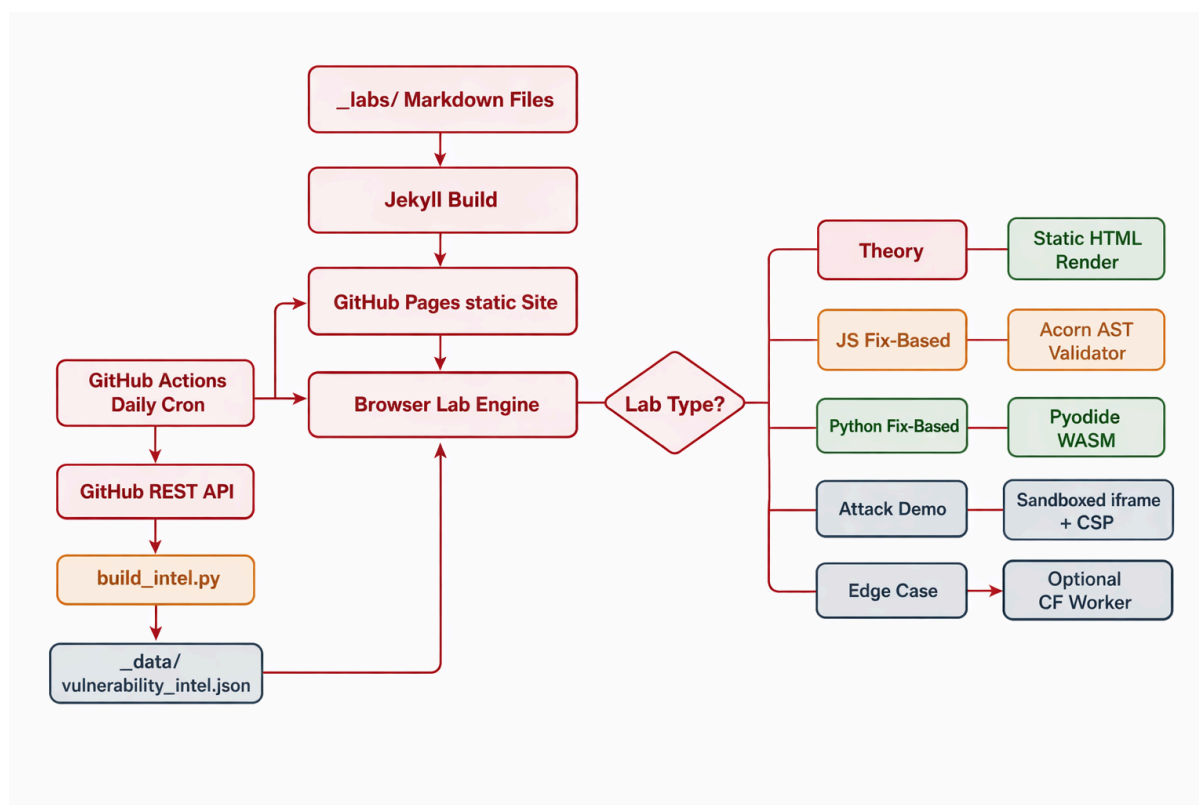
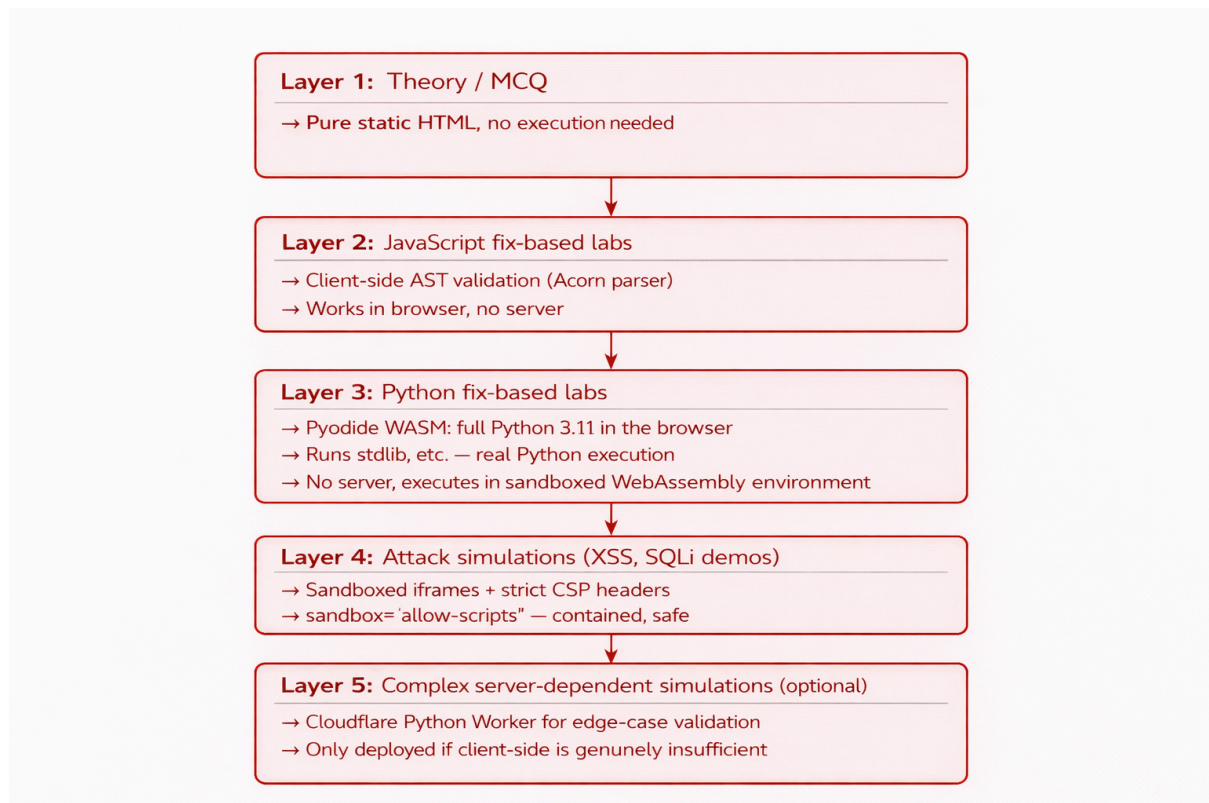


Fig : SYSTEM ARCHITECTURE

How Labs are Fetched: Labs are stored as Markdown files with YAML frontmatter in the `_labs/` collection of the BLT-University GitHub repository. Jekyll reads these files at build time and generates static HTML pages. No API calls, no database, no backend needed. If a user needs to fetch lab data at runtime (e.g., for dynamic search), the site uses the pre-built static JSON index (`_data/labs.json`) generated at build time.

How Labs are Rendered: Each lab file uses a `_layouts/lab.html` template that loads the client-side lab engine (JavaScript). The YAML frontmatter defines the lab metadata (title, difficulty, OWASP category, track, steps). The lab engine reads this data embedded in the HTML and drives the Identify → Explain → Fix interactive flow.

How Labs are working :

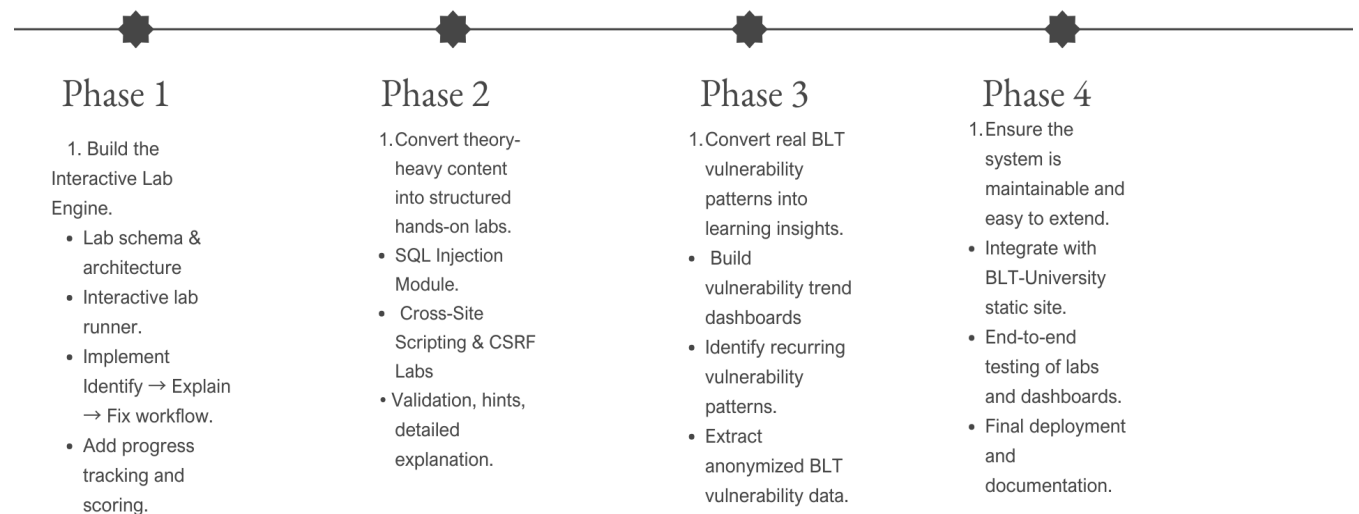


Timeline

The project will be implemented in four structured phases, gradually progressing from core system development to integration, testing, and deployment.



PROJECT PHASES AND TIMELINE



Timeline Detailed

Community Bonding Period (May 1 – May 24)

During the community bonding period, the primary focus is on deeply understanding the project, aligning with mentors, and preparing all foundational components required for a smooth and risk-free transition into the coding phase. This phase ensures that development begins with a clear, validated, and well-structured plan

- ❖ Introduction to mentors and the OWASP BLT community, establishing communication channels and aligning on expectations such as weekly updates, feedback cycles, and milestone tracking.
- ❖ Discussion with mentors to validate project scope, clarify technical decisions, and resolve any open design questions before implementation begins.
- ❖ Study of the existing _courses/ Markdown collection to understand Jekyll frontmatter structure, content organization, and rendering behavior.
- ❖ Core architecture decisions (lab schema, validation strategy, track structure) are finalized early to avoid downstream dependency issues.

- ❖ Detailed exploration of the BLT-University codebase, reviewing the existing `_courses/` collection structure and understanding how Jekyll frontmatter, layouts, and collections work.
- ❖ Designing the YAML frontmatter schema for the `_labs/` collection, defining all required fields: title, category, difficulty, owasp_category, track, steps, and validation config.
- ❖ Reviewing existing BLT lab content to map it to the designed schema and identify inconsistencies such as missing fields or structural mismatches.
- ❖ Refining the schema iteratively based on mentor feedback to ensure completeness and consistency before porting begins.
- ❖ Defining a strategy for handling complex content such as code snippets, embedded HTML, and images to ensure compatibility with static Jekyll rendering.
- ❖ Porting 2–3 sample BLT labs as static Markdown files to validate the schema end-to-end with Jekyll rendering.
- ❖ Validating generated Markdown using `python-frontmatter + jsonschema` and testing rendering with Jekyll to ensure correctness of schema and output.
- ❖ Designing the architecture of the interactive lab engine, defining the learning flow: Identify → Explain → Fix, along with state transitions and validation logic.
- ❖ Planning core security simulations (SQL Injection, XSS, IDOR), including inputs, outputs, execution flow, and safety boundaries to prevent misuse.
- ❖ Defining safe execution strategies (e.g., sandboxed environments or controlled evaluation logic) for running user-submitted code securely.
- ❖ Performing initial analysis of GitHub issues, pull requests, and discussions to evaluate availability and quality of vulnerability-related data for the intelligence pipeline.
- ❖ Identifying risks such as data inconsistencies, schema gaps, rendering limitations, and API constraints, along with defining mitigation strategies.
- ❖ Configuring `_config.yml` to register the `_labs/` collection and verifying that the Jekyll build pipeline works correctly in a local environment.

Week 1 (May 25 – May 31)

- Port all existing BLT labs as static Markdown files into the `_labs/` collection — one file per lab, with YAML frontmatter (title, category, difficulty, owaspcategory, track, steps).
- Define and finalize the YAML frontmatter schema for `_labs/` files, covering all required fields; share with mentors for review.
- Register the `_labs/` collection in `_config.yml` and verify that all lab files render correctly using the Jekyll build pipeline locally.
- Validate all generated lab files using `python-frontmatter + jsonschema` to ensure schema correctness.
- Set up the `validate-labs.yml` GitHub Actions workflow (triggered on PR to `_labs/`) to automatically check schema compliance and block merges on errors.

Week 2 (June 1 – June 7)

- Building UI components for the interactive lab engine including syntax-highlighted code blocks, selectable lines, and feedback indicators for user interaction.
- Implementing the “Identify” step where users select vulnerable lines, along with validation logic to verify correctness of selection.

- Designing a progressive hint system that provides step-by-step guidance when incorrect selections are made without directly revealing answers.
- Implementing the “Explain” step using structured multiple-choice questions with contextual explanations to reinforce conceptual understanding of vulnerabilities.
- Establishing state transitions between Identify → Explain → Fix to ensure a controlled and sequential learning flow.

Week 3 (June 8 – June 14)

- Implementing the “Fix” step where users submit corrected code and validating it against predefined logic to confirm vulnerability resolution.
- Designing a scoring system based on attempts and correctness to provide feedback and encourage learning progression.
- Implementing a localStorage-based persistence system to track user progress, completed labs, and scores across sessions. This local-first approach ensures that learners can continue their progress even after long gaps without requiring an account. The design is inspired by tools like Excalidraw, where data persists in the browser by default while still allowing optional integration with backend systems.
- When authentication (e.g., GitHub OAuth via BLT) is available, progress data can be associated with user identities and synchronized with backend services for cross-device persistence and reward integration.
- Integrating the full lab engine into `_layouts/lab.html` and ensuring seamless interaction between UI and logic.
- Writing unit tests for all state transitions and edge cases to ensure robustness of the interactive learning flow.

Week 4 (June 15 – June 21)

- Developing SQL Injection simulation using a mock query execution system to demonstrate how injected inputs manipulate database queries.
- XSS simulations will be executed inside sandboxed iframes with strict restrictions (`sandbox="allow-scripts"`) and a Content Security Policy (CSP) to prevent unintended script execution outside the lab environment.
- Creating IDOR simulation using a mock access control system to demonstrate unauthorized data access through parameter manipulation.
- Defining execution boundaries for all simulations to ensure safe and isolated behavior without exposing system vulnerabilities.
- Testing all simulations end-to-end to ensure correctness, usability, and educational clarity.
- Introduce learning track structure (Professional and Cyber Hygiene) and map labs to respective tracks.

Week 5 (June 22 – June 28)

- Integrate Pyodide for Python lab execution load WASM in browser, execute Python submissions fully client-side.

- Test Pyodide execution with Python lab content: verify correct fixes pass, vulnerable snippets fail, and execution is properly isolated.
- Validate client-side execution boundaries: ensure Python code cannot access browser globals outside the sandboxed Pyodide environment.
- Initial setup of optional Cloudflare Python Worker scaffold (workers/hint.py) structure only, full integration in Week 11.
- Verify at this stage that the system works fully without the Worker; confirm `_data/hints.json` fallback is correct.
- Finalizing Phase 1 deliverables including lab authoring system, lab engine, simulations, and client-side execution.

Week 6 (June 29 – July 5)

- Setting up GitHub Actions workflow to validate all lab files using `python-frontmatter` and `jsonschema` to ensure schema consistency across contributions.
- Fetching GitHub issues, pull requests, and discussions to collect vulnerability-related data for intelligence pipeline.
- Implement pattern-based security extraction logic using source-sink rule combinations and noise filtering.
- Generating structured JSON output (`vulnerability_intel.json`) for dashboard consumption.

Midterm Evaluation (July 6 – July 10)

- ❖ Delivering a fully functional prototype including migrated labs, interactive lab engine, simulations, and sandbox execution system.
- ❖ Demonstrating CI validation workflows and initial intelligence pipeline outputs.
- ❖ Reviewing progress with mentors and incorporating feedback for next phase improvements.

Week 7 (July 6 – July 12)

- Performing statistical analysis on collected vulnerability data to identify trends and frequently occurring issues.
- Structuring data to include category-wise frequency and time-based trends for dashboard visualization.
- July 6 - 8 reserved for midterm evaluation and mentor feedback. Development resumes from July 9.
- Optimizing dataset format for efficient frontend rendering and minimal payload size.
- Refining classification logic to improve accuracy and reduce noise in categorized data.

Week 8 (July 13 – July 19)

- Designing and implementing vulnerability dashboard UI using Chart.js for visualization of collected data.
- Refine learning tracks and expand track-specific lab coverage based on feedback.
- Creating bar charts and pie charts representing OWASP vulnerability distribution.
- Implementing filtering and search functionality for better data exploration.
- Adding contextual descriptions for each vulnerability category to improve understanding.

Week 9 (July 20 – July 26)

- Designing recommendation system to suggest labs based on user progress and vulnerability trends.
- Implementing a scoring algorithm combining frequency data, user progress, and OWASP category importance.
- Mapping vulnerability categories to corresponding labs for personalized learning recommendations.
- Displaying recommended labs dynamically on the dashboard with priority ranking.

Week 10 (July 27 – August 2)

- Designing and implementing learning analytics dashboard to track user progress across labs and categories.
- Building visual indicators such as completion charts and progress tracking metrics.
- Improving navigation and UX flow based on user interaction patterns observed during testing.
- Optimizing performance of dashboard rendering and data updates.

Week 11 (August 3 – August 9)

- Implementing streak tracking system to encourage consistent learning engagement.
- Building badge systems to reward milestones such as completing labs or achieving high scores.
- Implement badge system for completing OWASP categories (client-side, localStorage-based). Badge completion events will be structured to allow future integration with BLT's BACON reward system once backend authentication and APIs are available.
- Developing AI-powered hint systems using serverless API to provide contextual guidance to users.
- Implementing fallback mechanism for hint system to ensure reliability in case of API failure.

Week 12 (August 10 – August 16)

- Finalizing contributor documentation including lab creation guidelines and schema reference.
- Improving validation workflows to provide detailed error messages for contributors.

- Conducting security audit to identify potential vulnerabilities in frontend and sandbox system.
- Implementing accessibility improvements (WCAG 2.1 AA compliance, including keyboard navigation and focus management for all lab steps)

Final Phase (August 17 – August 24)

- ❖ Performing full end-to-end testing across all features including lab engine, simulations, dashboard, and recommendation system.
- ❖ Writing unit and integration tests for frontend components, the lab engine, and the CI validation pipeline.
- ❖ Recording demo video showcasing complete workflow and features of the platform.
- ❖ Finalizing documentation including architecture, setup guide, and contributor instructions.
- ❖ Submitting final deliverables and ensuring production readiness of the platform

FINAL SUBMISSION

As per GSOC timeline, (August 17 - 24 - 18:00 UTC)

- Final week: GSOC contributors submit their final work product and their final mentor evaluation (standard coding period)
- I will submit the final work product by 20th August 2026.

Timeline Summary

TIME PERIOD	TASKS
COMMUNITY BONDING PERIOD (1 MAY - 24 MAY)	
(May 1 - 7)	- Meet mentors, introduce myself - Discussing the plan of the project - Read the existing _courses/ Markdown files to understand how the Jekyll collection frontmatter is structured.
(May 8 - 15)	- Draft the YAML frontmatter schema for _labs/ files and share with mentors for review. - Map existing BLT labs to schema and identify missing fields or inconsistent fields - Identify handling strategy for images and complex HTML content - Discuss schema and design decisions with mentors and refine based on feedback - Draft structure for _layouts/lab.html (UI + rendering logic)
(May 15 - 24)	- Validate generated Markdown with Jekyll rendering - Perform initial analysis of GitHub issues/PRs to estimate available data for the intelligence pipeline - Identify any early risks and adjust implementation plan if needed

CODING PERIOD (25 MAY - 6 JULY)	
(WEEK 1) Migration & API Integration	
(May 25 - 26) Day 1 - 2	- Port all existing BLT labs as static Markdown files into the <code>_labs/</code> collection. - Finalize and validate the YAML frontmatter schema for <code>_labs/</code> files with mentors. - Register the <code>_labs/</code> collection in <code>_config.yml</code> and verify local Jekyll build. - Set up the <code>validate-labs.yml</code> GitHub Actions workflow.
(May 27 - 28) Day 3 - 4	- Generate YAML frontmatter for each lab
(May 29 - 30) Day 5 - 6	- Run full migration of all labs into <code>_labs/</code> - Validate Markdown using <code>python-frontmatter</code> - Handle edge cases (missing fields, formatting issues)
(May 31) Day 7	- Add <code>_labs/</code> collection to <code>_config.yml</code> - Integrate with Jekyll build - Verify labs render correctly on local site
(WEEK 2) Interactive Lab Engine (Core Flow)	
(June 1 - 2) Day 8 - 9	- Build UI components for code display and interaction - Highlight vulnerable code sections - Enable user selection of vulnerable lines
(June 3 - 4) Day 10 - 11	- Implement "Identify" step validation logic - Provide feedback for correct/incorrect selections - Add a hint system for guidance.
(June 5 - 6) Day 12 - 13	- Implement "Explain" step (MCQ-based questions) - Add contextual explanations for vulnerabilities - Provide reasoning feedback
(June 7) Day 14	- Implement basic state management for lab flow - Connect Identify → Explain → Fix transitions
(WEEK 3) Fix Step & Progress Tracking	
(June 8 - 9) Day 15 - 16	- Implement Fix step (code editor input / patch submission) - Validate user fixes using basic logic
(June 10 - 11) Day 17 - 18	- Implement scoring system for lab completion - Add feedback for correct/incorrect fixes

(June 12-13) Day 19 - 20	- Implement localStorage-based progress tracking - Store progress under blt-lab-progress key
(June 14) Day 21	- Integrate lab engine with _layouts/lab.html - Add Jest unit tests for state transitions
(WEEK 4)Security Simulations	
(June 15 - 16) Day 22 - 23	- Implement SQL Injection simulation - Build mock query execution logic - Display results dynamically
(June 17 - 18) Day 24 - 25	- Implement XSS simulation in sandboxed iframe - Demonstrate script injection safely
(June 19 - 20) Day 26 - 27	- Implement IDOR scenario (mock login system) - Simulate access control vulnerabilities
(June 21) Day 28	- Test all simulations - Fix inconsistencies and UI issues - Introduce learning track structure (Professional and Cyber Hygiene) and map labs to respective tracks.
(WEEK 5)Pyodide Integration & Optional Cloudflare Worker	
(June 22 - 23) Day 29 - 30	- Integrate Pyodide WASM for Python lab execution (client-side). Verify Python labs execute in browser without server.
(June 24 - 25) Day 31- 32	- Test Pyodide execution boundaries. Validate correct fixes pass, vulnerable snippets fail.
(June 26 - 27) Day 33 - 34	- Set up Cloudflare Python Worker (workers/hint.py). Verify system works 100% without Worker. Full Worker integration in Week 11.
(June 28) Day 35	- Finalize Phase 1: lab authoring system, lab engine, simulations, client-side execution. Final validation improvements.
(WEEK 6)CI Validation Pipeline & Intelligence Data Collection	
(June 29 - 30) Day 36 - 37	- Define JSON schema for lab metadata. Implement GitHub Actions workflow (PR trigger, python-frontmatter + jsonschema). - Authenticate with GitHub REST API (Actions token). Fetch GitHub issues (bug reports, security discussions) and PR discussion threads including CodeRabbit review comments.
(July 1 - 2) Day 38 - 39	- Write sync-labs.yml (daily cron 0 2 * * *, fail-safe on non-200). Implement OWASP Top 10 rule-based detection system. - Add fallback logic: if classifiable records < threshold, set dashboard_notice = "insufficient_data" to avoid misleading visualizations.

(July 3 - 4) Day 40 - 41	- Serialise output to <code>_data/vulnerability_intel.json</code> with schema: <code>{generated_at, total_comments_processed, categories[]}</code>. - Test end-to-end from API to Jekyll build. Validate JSON output schema.
(July 5) Day 42	- Write <code>build-intel.yml</code> workflow (daily cron). Document schema requirements and validation rules. Collect Phase 1 mentor feedback.
MIDTERM EVALUATION (6 JULY - 10 JULY)	
(WEEK 7)Vulnerability Analysis	
(July 6-8) Day 43 - 44	- Note: July 6 - 8 reserved for midterm evaluation and mentor feedback. Development resumes from July 9.
(July 8-9) Day 45 - 46	- Analyse vulnerability growth patterns. - Implement statistical analysis for vulnerability frequency. Track most common contributor mistakes.
(July 10 - 11) Day 47 - 48	- Generate monthly trend JSON embedded in <code>vulnerability_intel.json</code>. - Prepare datasets for visualization. Validate intelligence data quality. - Optimize dataset structure - Prepare visualization-ready data structures.
(July 12) Day 49	- Document data schema for dashboard consumers.
(WEEK 8)Vulnerability Dashboard	
(July 13 - 14) Day 50 - 51	- Design dashboard layout using Tailwind CSS. - Load intelligence JSON datasets using Fetch API.
(July 15) Day 52	- Implement bar charts (top 5 categories by count) - Implement pie chart (percentage breakdown).
(July 16 - 17) Day 53 - 54	- Add contextual explanations. - Add data freshness indicators and last-update timestamps. - Add filtering by OWASP category. Add search.
(July 18 - 19) Day 55 - 56	- Test dashboard responsiveness. - Optimize chart rendering performance. - Add caching strategies using <code>localStorage</code> and <code>sessionStorage</code>
(WEEK 9)Lab Recommendation System	
(July 20 - 21) Day 57 - 58	- Implement recommendation scoring formula: $\text{score} = 0.4 \times \text{freq_norm} + 0.4 \times \text{incompleteness} + 0.2 \times \text{owasp_weight}$. - Load <code>_data/owasp_weights.json</code>. Map vulnerability categories to lab

	slugs.
(July 22) Day 59	- Surface top-3 recommendations on learner dashboard. - Add notification badge for new recommendations.
(July 23) Day 60	- Tune weights, optimise performance.
(July 24 - 26) Day 61 - 63	- Document recommendation system architecture and algorithms. Generate Phase 2 completion report with metrics and achievements. - Collect mentor feedback and complete missed tasks.
(WEEK 10)Lab System Improvements + Learning Analytics	
(July 27 - 28) Day 64 - 65	- Create Figma design for learning analytics dashboard. - Design UI for tracking learning progress. Review design with mentors and finalize layout.
(July 29) Day 66	- Track completed labs using localStorage.
(July 30) Day 67	- Build progress charts using Chart.js. - Show vulnerability categories learned.
(July 31 - Aug 1) Day 68 - 69	- UX improvements: improve lab navigation flow. - Add progress indicators for each lab step.
(Aug 2) Day 70	- Test analytics system accuracy. Fix UI issues. Optimize rendering performance.
(WEEK 11)Learning Analytics + AI Assistant	
(Aug 3 - 4) Day 71 - 72	- Implement a learning streak system (daily streak tracker in localStorage). - Implement badge system for completing OWASP categories (client-side, localStorage-based).
(Aug 5 - 6) Day 73 - 74	- Integrate OpenAI gpt-4o-mini with prompt template.
(Aug 7) Day 75	- Write _data/hints.json static fallback. Test fallback triggers correctly on timeout/error.
(Aug 8) Day 76	- Playwright smoke tests for AI assistant UI.
(Aug 9)	- Mentor review iteration. Apply mentor feedback. Improve

Day 77	documentation clarity.
(WEEK 12)Contributor Ecosystem + Platform Security & Performance Review	
(Aug 10 - 11) Day 78 - 79	- Build a comprehensive workflow for end-to-end static site automation. Add error handling and retry mechanisms. Complete contributor lab-creation documentation (Markdown template, schema reference, workflow walkthrough).
(Aug 12 - 13) Day 80 - 81	- Improve validate-labs.yml error messages (line-specific schema error output). Ensure new labs follow schema rules. Platform security audit: review client-side code for vulnerabilities, prevent XSS injection from lab inputs, improve sandbox isolation.
(Aug 14) Day 82	- Code quality improvements: refactor JavaScript modules, improve maintainability of lab engine, simplify simulation logic.
(Aug 15) Day 83	- Performance optimization: DOM update optimization, improve lab loading speed.
(Aug 16) Day 84	- Accessibility improvements: keyboard navigation, ARIA labels, screen reader compatibility.
(WEEK 13-14)Final Testing & Deployment	
(Aug 17 - 18) Day 85 - 86	- Full Playwright cross-browser suite (Chrome, Firefox, Safari/WebKit). Jest full suite. Pytest full suite. Bug fixes.
(Aug 19 - 20) Day 87 - 88	- Validate simulation correctness. Edge-case testing for state machines.
(Aug 21 - 22) Day 89 - 90	- Record demo video: lab system, simulations, dashboard, recommendation engine, AI assistant.
(Aug 23 - 24) Day 91 - 92	- Final documentation review. Prepare a project report. Final mentor review. Final deployment verification. Submit work product by August 24.

OTHERS

Commitments

- During the entire GSoC period, I will be available and can dedicate around 40- 45 hours per week to the project, ensuring a minimum of 350+ hours.
- I do not plan to take up any internships or other major commitments during this period, so I can stay fully focused on the project.
- I will have my college end-semester exams from June 20 - June30. However, this has already been considered in my timeline, and I have kept buffer time to ensure it does not impact the project deliverables.
- To balance this period, I will adjust my schedule by contributing additional hours before and after the exam week to stay on track.
- I will complete tasks ahead of deadlines whenever possible to avoid delays and maintain steady progress.
- I will provide regular weekly updates to mentors and stay actively engaged for feedback and discussions throughout the project.

POST GSOC

- After GSoC, I plan to continue contributing to BLT-University by expanding the lab content and improving existing modules based on community feedback.
- I aim to extend the platform with additional vulnerability scenarios beyond the initial scope and improve the AI-assisted learning system using real user interactions. I will also focus on improving documentation and onboarding guides to help new contributors create and maintain labs more easily.
- Additionally, I plan to stay actively involved in the OWASP BLT community by reviewing contributions and supporting future contributors working on BLT.

LLM Usage

During the preparation of this proposal, ChatGPT, Copilot was used for supportive and editorial purposes.

Specifically, LLMs were used to:

- Improve clarity, grammar, and structure of the content
- Refine the phrasing of technical explanations
- Assist in organizing sections and formatting

All technical decisions, architecture design, and implementation planning reflect my own understanding. All generated content was carefully reviewed and validated to ensure accuracy and originality.